

Question 1

In the Customer Support Resolution Agent, the team finds that each response currently waits for three independent retrievals: get_customer for account context, a support-history lookup, and a related FAQ search. Running them one after another averages 4.1 seconds, while each retrieval typically finishes in about 1.1–1.4 seconds on its own. Which orchestration pattern best reduces latency for this workflow?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Routing — Send the inquiry to a path that retrieves only the information sources predicted to be relevant for that request type

Evaluator-Optimizer — Retrieve the most likely source first, evaluate whether the answer is sufficient, then request more sources only if needed

Prompt Chaining — Combine the three retrieval needs into one LLM step that asks Claude to gather and synthesize the needed context

Parallelization with sectioning — Run the independent retrieval tasks concurrently and combine their outputs through an aggregator ✓

ANSWER

The correct option is **Parallelization with sectioning — Run the independent retrieval tasks concurrently and combine their outputs through an aggregator.**

This workflow has three independent I/O bound retrievals that are all needed for each response. When run one after another their durations add up to about 4.1 seconds. Running them at the same time reduces the wall clock latency to roughly the longest single call plus a small aggregation overhead. With typical durations of about 1.1 to 1.4 seconds for each retrieval, concurrency brings the total close to that range rather than the sum. An aggregator can then combine the results into a single context package for the model, which is exactly what this approach targets.

Routing — Send the inquiry to a path that retrieves only the information sources predicted to be relevant for that request type is not suitable because this workflow consistently needs all three sources for each answer. Routing aims to skip whole branches for some requests, yet here you would risk missing essential context or still end up executing multiple branches in sequence. It does not address the core latency problem when every branch is required.

Evaluator-Optimizer — Retrieve the most likely source first, evaluate whether the answer is sufficient, then request more sources only if needed is not ideal because it introduces extra decision steps and keeps work sequential. Since the agent typically needs customer context, history, and FAQ data together, this pattern increases tail latency and spends time on checks rather than finishing all needed retrievals at once.

Prompt Chaining — Combine the three retrieval needs into one LLM step that asks Claude to gather and synthesize the needed context does not actually remove the external I O time. The model still cannot access those systems without tool calls, and placing everything into one model prompt risks hallucinations or missing data. It also centralizes the work into a single longer step rather than overlapping the independent retrievals.

EXAM TIP

When multiple steps are *independent* and all are *required* for each answer, choose to *parallelize* so that total time approaches the *slowest branch* instead of the sum of all branches.

REFERENCES

Check out these links for more details:

- [AWS Step Functions - Parallel state](#)
- [AWS Well-Architected Framework - Performance Efficiency Pillar](#)
- [AWS Step Functions - Map state for parallel workloads](#)

Question 2

A developer productivity agent is asked to add comprehensive tests to an unfamiliar legacy codebase, but the team does not yet know which modules carry the most risk or how dependencies connect. Which task decomposition approach fits this situation?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

Map the codebase structure first, identify risk-heavy and high-impact areas, then build a prioritized testing plan that evolves as dependencies are uncovered ✓

Create tests one file at a time by walking the repository tree in directory order until every file has coverage

Ask Claude to generate the complete test suite in one large request after a brief repository scan

Focus testing only on files changed recently and defer older legacy modules unless failures appear

ANSWER

The correct option is **Map the codebase structure first, identify risk-heavy and high-impact areas, then build a prioritized testing plan that evolves as dependencies are uncovered.**

This approach fits an unfamiliar legacy codebase because it starts with discovery. You first build a mental and visual model of modules, boundaries, and dependencies, then you assess which areas are most critical by risk and impact. That lets you design a prioritized and iterative testing plan that adapts as you learn more about hidden couplings and operational constraints. This is essentially risk based and dependency aware decomposition, which is what you need when you do not yet know where the dangers are or how modules interact.

Create tests one file at a time by walking the repository tree in directory order until every file has coverage is incorrect because directory order does not reflect architectural risk or dependency importance. This linear sweep often wastes effort on low value code while deferring the highest risk paths and integration seams that should be tested first.

Ask Claude to generate the complete test suite in one large request after a brief repository scan is incorrect because a one shot generation ignores the need for progressive learning and refinement. Without a deep understanding of side effects, contracts, and hidden dependencies, bulk generated tests will be shallow or brittle and they will miss the highest risk behaviors that emerge only after mapping and analysis.

Focus testing only on files changed recently and defer older legacy modules unless failures appear is incorrect because recent change is not a reliable proxy for risk in a legacy system. Some of the most critical and fragile code is old and rarely touched, and it often sits on key integration points where failures are costly. Limiting effort to recent changes leaves significant unknown risks unaddressed.

EXAM TIP

When the problem space is unclear, start with discovery and prioritize by *risk* and *dependencies* rather than by directory order or recent edits. Build tests iteratively so that each round informs the next.

Question 3

In the developer productivity agent, an MCP tool searches an external code reference index that returns results in pages of 50. Logs show many searches match more than 200 entries, and the current implementation automatically retrieves every page, adding 15–20 seconds of latency. How should the tool's pagination behavior be redesigned?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

Split the workflow into separate search references and fetch additional references tools

Add server-side relevance scoring and return only the 50 highest-ranked references

Add a max_pages input with a default of 2 to limit how many pages the tool retrieves internally

Return the first page along with the total match count and a cursor for later pages ✓

ANSWER

The correct option is **Return the first page along with the total match count and a cursor for later pages.**

Return the first page along with the total match count and a cursor for later pages implements client driven pagination. This design eliminates unnecessary latency by fetching only the first 50 results immediately and giving the caller the information and handle needed to decide whether to request more. Providing the total match count sets clear expectations, and the cursor enables efficient, incremental retrieval of subsequent pages only when the agent or user actually needs them. This mirrors common cloud API patterns where a token indicates there are more results and the client can continue as required, which keeps interactions responsive and avoids hidden long running calls.

This approach is also operationally safer because it bounds default latency and cost while preserving completeness on demand. Many AWS APIs demonstrate this model with continuation tokens and flags that indicate whether results are truncated, which shows it is a proven pattern for scalable list operations.

Split the workflow into separate search references and fetch additional references tools is incorrect because changing the number of tools does not address the root cause of latency. The eager internal pagination would still occur and the system would still suffer from long waits and extra calls, while the added tool complexity would not improve performance.

Add server-side relevance scoring and return only the 50 highest-ranked references is incorrect because the question asks how to redesign pagination behavior rather than ranking. You often cannot change the external service and even if you could, forcing only the top 50 removes the ability to retrieve more when needed and does not provide a cursor or total count for controlled follow up requests.

Add a `max_pages` input with a default of 2 to limit how many pages the tool retrieves internally is incorrect because any fixed cap still performs hidden background fetches that add latency, and it risks truncating useful results arbitrarily. It is better to return a cursor and let the caller intentionally request more pages so that work is done only when it is needed.

EXAM TIP

When pagination causes high latency, prefer designs that return an initial page with a *cursor* and a *total count* so the caller decides whether to fetch more. Look for patterns like continuation tokens or next tokens in AWS services to recognize the best choice.

REFERENCES

Check out these links for more details:

- [Using pagination in the AWS CLI](#)
- [Amazon S3 ListObjectsV2 API with ContinuationToken pagination](#)
- [DynamoDB Scan pagination with LastEvaluatedKey](#)

Question 4

In the Claude Code workflow, the team maintains a `/migration` skill that creates database migration files and uses `$ARGUMENTS` for the migration name. During production use, developers sometimes invoke it without a name, the skill occasionally carries schema assumptions from earlier conversations, and one run triggered destructive test cleanup because the skill had broad tool access. Which configuration approach best addresses these issues?

This question deals with the [Code Generation with Claude Code Exam Scenario](#).

(Choose 1)

Replace `$ARGUMENTS` with positional inputs such as `$1` and `$2`, reference the intended schema files with `@` syntax, and add a frontmatter description that warns about destructive actions

Add argument-hint in the skill frontmatter, set context: fork for isolated execution, and constrain allowed-tools to file-writing operations ✓

Separate the workflow into `/migration-create` and `/migration-apply` skills, include instructions to ask for the migration name when absent, and assign different allowed-tools scopes to each skill

Add validation instructions in `SKILL.md` for checking that `$ARGUMENTS` contains a valid name, tell the skill to ignore earlier conversation context, and list operations that should not be performed

ANSWER

The correct option is **Add argument-hint in the skill frontmatter, set context: fork for isolated execution, and constrain allowed-tools to file-writing operations.**

This configuration addresses all three failure modes directly. An argument-hint in the skill frontmatter makes the expected input obvious and encourages or prompts developers to supply the migration name, which reduces the chance of blank or malformed invocations. Setting context: fork runs the skill in an isolated branch of the conversation so prior dialogue cannot bleed schema assumptions into the current execution. Constraining allowed-tools to file-writing operations narrows the capability surface so the skill can generate migration files without having the broad access that enabled destructive test cleanup or other unintended side effects.

Replace `$ARGUMENTS` with positional inputs such as `$1` and `$2`, reference the intended schema files with `@` syntax, and add a frontmatter description that warns about destructive actions is not sufficient because positional inputs do not enforce that a name is provided and a description only warns rather than constrains behavior. Referencing files with `@` can help share context yet it does not isolate the run or prevent dangerous tool use.

Separate the workflow into `/migration-create` and `/migration-apply` skills, include instructions to ask for the migration name when absent, and assign different allowed-tools scopes to each skill relies on conversational prompts to gather missing input and still does not guarantee isolation from prior

context. Splitting skills can reduce risk but it does not ensure the tight file-only restriction, and without context: fork the schema carryover issue can persist.

Add validation instructions in SKILL.md for checking that \$ARGUMENTS contains a valid name, tell the skill to ignore earlier conversation context, and list operations that should not be performed is guidance rather than enforcement. The model may overlook instructions, the conversation can still leak context without isolation, and broad tool access remains unless it is explicitly constrained.

EXAM TIP

When a prompt mentions missing inputs, context leakage, and risky tool effects, look for a configuration that both *enforces* inputs and *sandboxes* execution. Features like *argument-hint*, *context: fork*, and narrowly scoped *allowed-tools* provide concrete guardrails that instructions alone cannot.

Question 5

The team is extending the structured data extraction pipeline to pull a recurring clause type from a 10-year archive of unstructured documents totaling about 5 million tokens. Each document is roughly 5,000 tokens, and Claude's context window is about 200,000 tokens, so the full archive cannot fit in one request. What approach should the pipeline use?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Process only the first documents that fit within the available context and leave the rest of the archive out

Increase max_tokens to the context-window size and submit the archive while relying on Claude to handle the workload

Concatenate every document into a single request and allow the prompt to exceed the context window

Divide the archive into document-level or clause-level chunks, extract incrementally through batch processing or staged turns, and combine the validated JSON outputs ✓

ANSWER

The correct approach is **Divide the archive into document-level or clause-level chunks, extract incrementally through batch processing or staged turns, and combine the validated JSON outputs**. This is the only option that can cover a 5 million token archive when the model context is about 200,000 tokens while preserving accuracy and completeness.

This chunking and batching strategy works because it keeps each request within the context window while still processing the entire collection. You can split at document or clause boundaries to maintain semantic coherence, then run extraction jobs in batches or across staged turns. Each batch produces structured and validated JSON, ideally with consistent schemas and stable identifiers, which you then merge deterministically at the end. This pattern supports retries, parallelization, and auditability, and it avoids truncation or context overflows that would otherwise cause failures or hallucinations.

Process only the first documents that fit within the available context and leave the rest of the archive out is incorrect because it intentionally omits a large portion of the data and therefore cannot achieve complete recall of the clause type across the archive. Any solution that discards data will bias results and undermine reliability.

Increase max_tokens to the context-window size and submit the archive while relying on Claude to handle the workload is incorrect because max_tokens controls the maximum length of the model's output and it does not expand the allowable input context. The input must still remain within the model's context window, so the archive would not fit and the request would fail or be truncated.

Concatenate every document into a single request and allow the prompt to exceed the context window is incorrect because inputs that exceed the context limit are not accepted by the API. Even approaching the limit with massive concatenation would degrade performance and increase the risk of partial processing or lost information.

EXAM TIP

When total input size exceeds the context window, look for designs that *chunk*, *batch*, and *merge structured outputs*. Be cautious of options that try to push more input by changing *max_tokens* or by concatenating everything, since the context limit still applies.

REFERENCES

Check out these links for more details:

- [Anthropic docs: Structured outputs and JSON schema](#)

Question 6

A developer asks the agent to find untested execution paths in a legacy authentication component spread across dozens of files. After using Read on several source files, the agent's later responses become less consistent, it loses track of earlier code patterns, and it still has not found all related tests or traced the main login flows. What approach should the team use to finish the investigation?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

Capture the current observations in a concise report, reset the context entirely, and continue the investigation using only that report as the reference

Launch focused subagents for targeted tasks such as locating authentication tests and tracing login-flow dependencies, while the main agent coordinates their results and maintains the overall investigation state ✓

Use /clear to reset context, re-read only the files that seemed most important, and store key notes in a scratchpad file that survives later resets

Stop reading full files and rely primarily on Grep searches for known function names to reduce how much code is loaded into context during the remaining exploration

ANSWER

The correct approach is **Launch focused subagents for targeted tasks such as locating authentication tests and tracing login-flow dependencies, while the main agent coordinates their results and maintains the overall investigation state**. This choice counters context loss and inconsistency by assigning narrow, well scoped objectives to specialized workers while a coordinating agent preserves the global state of the investigation and synthesizes results.

This approach reduces pressure on any single context window because each worker focuses on a bounded slice of the codebase such as test discovery, dependency mapping, or path tracing. The coordinator maintains a structured investigation record that includes a checklist of target flows, a map of files and functions, and a log of findings with links back to source lines. As results arrive, the coordinator merges them, resolves conflicts, and issues follow ups to close gaps. This preserves continuity across many tool calls and file reads, improves recall of earlier patterns, and allows parallel progress without losing the thread of the main authentication flows.

In practice the team can spin up workers to enumerate all authentication related tests, to trace the main login entry points through helpers and middleware, and to catalog dependencies such as configuration, feature flags, and user stores. The coordinator can enforce coverage criteria like ensuring each identified entry point has a traced path and corresponding tests, and can request targeted code reads only where evidence is missing. This disciplined orchestration finishes the investigation systematically and avoids repeating partial reads that drift over time.

The option *Capture the current observations in a concise report, reset the context entirely, and continue the investigation using only that report as the reference* is incorrect because a single summary cannot retain enough code level detail to drive precise path tracing across dozens of files. It also discards useful intermediate artifacts and tool outputs, which leads to rework and new blind spots rather than better coverage.

The option *Use /clear to reset context, re-read only the files that seemed most important, and store key notes in a scratchpad file that survives later resets* is incorrect because it repeats the same single agent limitation and introduces selection bias toward what already looked important. A scratchpad helps, yet it does not solve the core problem of distributing work and maintaining a coherent, continuously updated global state across many targeted reads.

The option *Stop reading full files and rely primarily on Grep searches for known function names to reduce how much code is loaded into context during the remaining exploration* is incorrect because search by known names misses indirect calls, framework wiring, configuration driven flows, and negative or edge cases. It also loses surrounding control flow that is essential for accurate path tracing and test gap identification.

EXAM TIP

When the codebase is large and the agent starts forgetting earlier details, favor a *divide and conquer* pattern with a coordinating agent and specialized workers so each keeps a narrow context while the coordinator preserves overall state and coverage.

Question 7

In the structured data extraction pipeline, tool_use with a strict JSON schema has made every Claude response parse correctly, but some extracted records still have totals that do not reconcile and amounts placed in the wrong fields. What should the team understand and add to the design?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Treat the schema behavior as a broken guarantee and report a defect in tool use

Recognize that schemas prevent syntax failures, then add semantic validation such as `calculated_total`, `stated_total`, and `conflict_detected` fields ✓

Redesign the schema so every extracted value is accepted as an unconstrained string

Repeat the same extraction request until the returned values eventually reconcile

ANSWER

The correct option is **Recognize that schemas prevent syntax failures, then add semantic validation such as `calculated_total`, `stated_total`, and `conflict_detected` fields**. This approach acknowledges that a strict JSON schema ensures the response parses yet does not guarantee that the extracted values are logically consistent, and it adds explicit checks and fields to detect and communicate reconciliation issues.

This option is correct because a strict schema gives you structural guarantees such as required fields, types, and formats, which prevents parse and syntax failures. However, business rules like totals matching the sum of line items or amounts appearing in the correct fields are semantic constraints. The design should therefore compute cross field validations such as a calculated total from line items, capture the stated total from the document, compare them, and then set a conflict flag when they diverge. Adding fields like `calculated_total`, `stated_total`, and `conflict_detected` supports downstream auditing and remediation, and it enables deterministic handling such as routing conflicted records to review or triggering retries with clarified prompts.

Treat the schema behavior as a broken guarantee and report a defect in tool use is incorrect because the schema is functioning as intended. Schemas enforce structure and types and do not assert that business logic is satisfied, so there is no defect to report when totals fail to reconcile.

Redesign the schema so every extracted value is accepted as an unconstrained string is incorrect because removing constraints discards valuable structural validation. That change would increase parse success at the cost of allowing more silent data quality issues and it would make semantic validation even harder.

Repeat the same extraction request until the returned values eventually reconcile is incorrect because retries do not introduce new semantic checks. Blind repetition increases latency and cost and still lacks the explicit cross field validation that is needed to detect and explain mismatches.

EXAM TIP

Differentiate *syntactic* validation from *semantic* validation. Schemas keep structure correct, while business rules need explicit cross field checks and flags that reveal mismatches for downstream handling.

REFERENCES

Check out these links for more details:

- [Anthropic docs: Tool Use](#)

Question 8

The structured data extraction pipeline must produce JSON fields such as document title, effective date, monetary amounts, and category tags, but source documents use inconsistent formats like dates written as “March 4, 2025” or “2025-03-04” and tags shown as icons or prose. What is the most reliable design for consistent structured output?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Run a separate Claude extraction request for each output field to standardize handling by field type

Return each value exactly as it appears in the source and normalize formats only in downstream code

Generate multiple extraction results for each document and keep the format that appears most often

Define a strict JSON schema for the extraction output and include explicit format normalization rules in the prompt ✓

ANSWER

The correct option is **Define a strict JSON schema for the extraction output and include explicit format normalization rules in the prompt**. This design enforces a single structure and consistent formatting at generation time, so dates, amounts, and category tags are standardized no matter how the source is written.

This choice is right because a strict schema defines required fields, types, and allowable values, and the prompt tells the model exactly how to convert varied inputs into one canonical form. For example you can require dates in ISO 8601 format such as 2025-03-04, represent monetary amounts as numbers with a separate currency code, and select category tags only from a controlled list. You can also specify how to map icons or prose into those tags and what to do when a value is missing or ambiguous, which yields predictable JSON that downstream systems can validate and trust.

A well defined schema and normalization rules also improve testability and monitoring. You can validate outputs against the schema, catch deviations early, and evolve the schema with versioning while keeping consumers stable. This reduces rework in downstream code and avoids error prone format guessing later in the pipeline.

Run a separate Claude extraction request for each output field to standardize handling by field type is not reliable because it fragments context and increases latency and cost, and it creates risk of contradictions across fields. Cross field dependencies such as dates tied to effective periods or tags derived from the title are harder to resolve when fields are extracted in isolation.

Return each value exactly as it appears in the source and normalize formats only in downstream code pushes ambiguity downstream and leads to inconsistent data contracts. It makes validation harder, it complicates mapping icons or prose into controlled tags, and it increases failure modes when multiple formats appear. Consistency is best achieved at generation time, not after the fact.

Generate multiple extraction results for each document and keep the format that appears most often does not guarantee correctness because popularity is not a standard. Majority voting can entrench the wrong format, it wastes compute, and it still fails to enforce required fields, types, and enumerations that a schema would provide.

EXAM TIP

When options mention enforcing *structure* at generation time with a strict *schema* and clear instructions to *normalize* dates, numbers, and tags, they are usually right. Look for language about required fields, formats like ISO 8601, controlled vocabularies, and validation against a schema.

REFERENCES

Check out these links for more details:

- [Anthropic docs: Structured Outputs](#)
- [Anthropic docs: Prompt engineering best practices](#)
- [JSON Schema official website](#)

Question 9

In the structured data extraction system, a required JSON schema field for counterparty address remains null for a batch of documents after three retry-with-error-feedback attempts. Each retry includes the original document, the prior extraction, and the validator's specific missing-field error. What should the team examine before queuing more retries?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Raise the retry budget from 3 to 5 so the extraction has additional chances to satisfy validation for difficult fields

Inspect whether the counterparty address is present in the provided document or only referenced as available in a separate attachment ✓

Move the retry attempts to a higher-capacity model tier in case the current model is struggling with this field

Change the JSON schema so the counterparty address field is nullable and downstream systems can continue without a value

ANSWER

The correct answer is **Inspect whether the counterparty address is present in the provided document or only referenced as available in a separate attachment**. This is the action that best explains persistent null values after multiple guided retries, and it directly targets whether the model has access to the needed signal.

When a required field remains empty after several retries that include the original document, the prior extraction, and a targeted validator error, the most likely root cause is that the information is not actually visible to the model. The team should verify that the counterparty address exists in the primary document content or that any referenced attachment containing the address is included in the input payload. If the address lives only in an attachment that is not passed to the model, then no number of retries will succeed. The remediation is to adjust ingestion to include the attachment, expand context windows, or fetch the address from a trusted system of record before attempting more retries.

Raise the retry budget from 3 to 5 so the extraction has additional chances to satisfy validation for difficult fields is incorrect. Increasing retries provides diminishing returns when the underlying issue is missing or out of scope input. Extra attempts add latency and cost without increasing the probability of success if the address is not accessible to the model.

Move the retry attempts to a higher-capacity model tier in case the current model is struggling with this field is incorrect. A larger or higher capacity model cannot extract information that is not present

in its inputs. Model upgrades can help with ambiguous formatting or complex layouts, but they will not solve a visibility gap where the field only exists in an unprovided attachment.

Change the JSON schema so the counterparty address field is nullable and downstream systems can continue without a value is incorrect. Making a required field nullable sidesteps the validation contract and can push data quality issues downstream. The right first step is to confirm that the source data is available and correctly provided to the extraction step, rather than weakening the schema.

EXAM TIP

When a *required* field repeatedly fails extraction, first confirm the signal exists in the inputs. Check document coverage, attachments, and preprocessing before adding more retries or changing models.

REFERENCES

Check out these links for more details:

- [Amazon Textract documentation overview](#)
- [AWS Step Functions error handling and retries](#)

Question 10

While validating the customer support resolution agent, the team finds that prompt guidance about the \$500 autonomous refund limit occasionally still allows a \$650 process_refund call. Which enforcement design should the team use?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Add a tool-call interception hook that rejects process_refund requests over \$500 and routes the case to escalate_to_human ✓

Put the \$500 refund rule at the beginning of the system prompt and restate it in each customer request

Reduce the model temperature so the agent is less likely to deviate from the refund policy

Update the process_refund tool description to mention that amounts above \$500 require human escalation

ANSWER

The correct option is **Add a tool-call interception hook that rejects process_refund requests over \$500 and routes the case to escalate_to_human**. This choice guarantees that any attempt to issue a refund above the allowed limit is blocked at execution time and that the case is programmatically handed off for human review.

An interception hook or middleware that validates tool inputs before execution creates a deterministic control point. It prevents out-of-policy actions even when the model produces a mistaken tool call and it ensures consistent enforcement across all conversations. This design also supports auditable outcomes since every blocked call and routed escalation can be logged and traced. Prompting can still provide guidance, yet the final enforcement must live at the tool boundary to be reliable.

Put the \$500 refund rule at the beginning of the system prompt and restate it in each customer request is incorrect because prompt instructions reduce risk but do not guarantee compliance. The model can still generate an out of policy tool call, so relying only on instructions leaves a gap in enforcement.

Reduce the model temperature so the agent is less likely to deviate from the refund policy is incorrect because temperature mainly adjusts variability and does not provide a policy guarantee. A low temperature can still produce tool calls that exceed the limit, so it is not an enforcement mechanism.

Update the process_refund tool description to mention that amounts above \$500 require human escalation is incorrect because tool descriptions guide the model but do not block execution. Descriptions can improve adherence but they cannot ensure that an invalid call will be rejected without a programmatic validator.

EXAM TIP

When a scenario asks how to prevent an out of policy tool call, choose a programmatic guard that validates inputs at execution time. Use prompts for guidance, yet *enforce* rules with middleware, validators, or guardrails at the tool boundary.

REFERENCES

Check out these links for more details:

- [Guardrails for Amazon Bedrock](#)
- [Agents for Amazon Bedrock overview](#)
- [Agents for Amazon Bedrock action groups](#)

Question 11

In the developer productivity agent, a targeted Edit against a legacy source file fails because the selected anchor text occurs multiple times and cannot be matched uniquely. What should the agent do next?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

Remove the extra repeated blocks first so the remaining anchor text is unique

Run the same Edit call again with the identical anchor text and replacement

Use Bash with a stream-editing command such as sed to patch the file

Use Read to load the whole file, then Write back the fully revised content ✓

ANSWER

The correct option is **Use Read to load the whole file, then Write back the fully revised content**. When the selected anchor text appears in multiple places and cannot be matched uniquely, a targeted edit is not deterministic. The safe response is to load the entire file, apply the intended change with full context, and then write the consolidated result in a single update.

This approach works because targeted edit operations depend on a unique anchor to guarantee an exact and reproducible change. If the anchor is ambiguous, the tool cannot ensure it is modifying the correct region. Reading the whole file allows the agent to reason over broader context, select the right occurrence programmatically, and apply any related refactors together. Writing back the complete revised content also avoids partial or conflicting updates and preserves a clear before and after state.

Remove the extra repeated blocks first so the remaining anchor text is unique is incorrect because you cannot reliably remove duplicates without a trustworthy way to target them. Acting on ambiguous matches risks deleting the wrong sections and can compound errors across multiple edits.

Run the same Edit call again with the identical anchor text and replacement is incorrect because it replays the same failing conditions. If the tool could not resolve the anchor uniquely once, repeating the identical request will not provide new information and will likely fail again.

Use Bash with a stream-editing command such as sed to patch the file is incorrect because it relies on pattern matching that suffers from the same ambiguity and increases the risk of overmatching or corrupting structure. It also bypasses the agent's built in editing guarantees and makes the change less traceable and less deterministic.

EXAM TIP

If a targeted edit cannot find a *unique anchor*, think escalation. Prefer loading the full artifact to gain context and then write the finished result so the change is *deterministic*. Avoid retrying the same failing edit or switching to shell tools unless the scenario explicitly requires it.

Question 12

In the Multi-Agent Research System, the web search subagent calls `search_web` for a topic and receives `{"results": [], "status": "success"}`, so it tells the coordinator that no relevant sources were found. Investigation shows the search API was unavailable and emitted empty responses for every query. How should the MCP tool's error handling be revised to avoid treating outages as empty search results?

This question deals with the [Multi-Agent Research Exam Scenario](#).

(Choose 1)

Populate the results array with fallback content whenever the live search returns no matches

Signal backend access failures with the MCP `isError` flag, and use empty successful responses only when a completed search finds no matches ✓

Add a confidence field to the response so the subagent can estimate whether an empty result represents a real outcome

Retry automatically inside the MCP tool so the subagent sees an empty result only after the retry sequence finishes

ANSWER

The correct option is **Signal backend access failures with the MCP `isError` flag, and use empty successful responses only when a completed search finds no matches.**

This choice is correct because the tool must clearly separate transport or backend failures from domain outcomes. When the search API is unavailable or returns a malformed or empty payload due to an outage, the MCP tool should return a result with the `isError` flag set so the coordinator can treat it as a failure and decide whether to retry, switch providers, or escalate. An empty results array should be reserved for a successful call that completed correctly and simply found no matches. This preserves a clean contract where errors control flow and data represents only completed outcomes, which prevents the subagent from misclassifying outages as legitimate empty searches.

Populate the results array with fallback content whenever the live search returns no matches is incorrect because fabricating or injecting fallback items into the results blurs the line between real search outcomes and error states. It risks downstream hallucinations and prevents the coordinator from recognizing an outage and applying the correct retry or failover policy.

Add a confidence field to the response so the subagent can estimate whether an empty result represents a real outcome is incorrect because confidence scores do not reliably indicate transport or backend failure. They also conflate semantic relevance with system health. Backend outages should be signaled explicitly as errors rather than hinted through metadata on a successful response.

Retry automatically inside the MCP tool so the subagent sees an empty result only after the retry sequence finishes is incorrect because hiding failures behind internal retries still collapses distinct

states into a single empty outcome and removes centralized control over retry budgets and backoff. Retries can be useful, yet they should surface an error if the backend remains unavailable so the orchestrator can apply consistent policies across tools.

EXAM TIP

When deciding between an *error* and an empty result that indicates *no matches*, ask whether the call truly completed. If the system did not complete the operation, surface an error so orchestration can retry or fail over. Use empty results only for successful calls that returned zero hits.

REFERENCES

Check out these links for more details:

- [Model Context Protocol official site](#)
- [Model Context Protocol specification on GitHub](#)

Question 13

In the multi-agent research system, the synthesis agent is preparing a report on the impact of AI on employment and receives conflicting statistics from two credible sources: a 2024 McKinsey cross-industry survey reports that 30% of jobs will be affected, while a 2025 OECD developed-country GDP-based analysis reports that 14% of jobs will be affected. How should the agent represent this conflict in the final report?

This question deals with the [Multi-Agent Research Exam Scenario](#).

(Choose 1)

Use only the OECD value because its report is more recent, treating newer publication dates as superseding older figures

Include both percentages with source attribution, publication dates, and a note explaining the different methodologies behind the estimates ✓

Exclude both percentages because they conflict, and replace them with a general statement that employment impacts are expected

Combine the two estimates into a single averaged figure of 22% as a neutral representation across the sources

ANSWER

The correct option is **Include both percentages with source attribution, publication dates, and a note explaining the different methodologies behind the estimates.**

This approach is right because readers need visibility into credible but divergent findings and the reasons they differ. The report should present the 30 percent figure from the 2024 McKinsey cross industry survey and the 14 percent figure from the 2025 OECD developed country GDP based analysis with clear citations and dates. It should then briefly explain that the McKinsey result comes from a cross industry survey while the OECD result is bounded by developed country scope and a GDP based framing. These design choices change the denominator, the sampling frame, and the interpretation of what affected means. By keeping both numbers and explaining methodology and scope, the synthesis conveys uncertainty and helps decision makers understand why estimates vary.

Use only the OECD value because its report is more recent, treating newer publication dates as superseding older figures is incorrect because recency alone does not resolve methodological differences. The underlying scope and analytic approach drive the gap in estimates, so discarding a credible earlier source would remove useful context.

Exclude both percentages because they conflict, and replace them with a general statement that employment impacts are expected is incorrect because it hides decision critical quantitative signal. A general statement lacks transparency and does not allow readers to gauge the plausible range of impact or the reasons for divergence.

Combine the two estimates into a single averaged figure of 22% as a neutral representation across the sources is incorrect because averaging non comparable estimates implies false precision and erases methodological differences. An average can be misleading when the sources use different populations, definitions, and measurement strategies.

EXAM TIP

When sources conflict, *attribute* each figure, include *publication dates*, and explain *methodology* and scope differences. Preserve the range rather than trying to *average* or choose only the newest number.

Question 14

In the Claude Code workflow, the team wants to add a /dependency-audit skill that inspects the repository for risky dependencies and code patterns. It must be shared with all project developers, limited to read-only discovery tools, and run its lengthy analysis without filling the main conversation. Which configuration best fits these requirements?

This question deals with the [Code Generation with Claude Code Exam Scenario](#).

(Choose 1)

Document the audit workflow in CLAUDE.md so it is loaded as part of the project's default instructions

Create the skill under ~/.claude/skills/ with frontmatter that sets allowed-tools to Read, Grep, and Glob and uses context: fork

Create a project command under .claude/commands/ and configure allowed-tools to Read, Grep, and Glob

Create the skill under .claude/skills/ with a SKILL.md that includes allowed-tools: ["Read", "Grep", "Glob"] and context: fork ✓

ANSWER

The correct option is **Create the skill under .claude/skills/ with a SKILL.md that includes allowed-tools: ["Read", "Grep", "Glob"] and context: fork.**

Placing the skill in the project at .claude/skills/ ensures it is version controlled and shared with every developer who works on the repository. Defining the frontmatter in SKILL.md with allowed-tools set to Read, Grep, and Glob constrains the skill to read only discovery actions because it can read files and search patterns yet cannot write or execute. Setting context: fork runs the lengthy dependency audit in a separate forked context so it does not crowd the main conversation while still providing an isolated thread for progress and results.

Document the audit workflow in CLAUDE.md so it is loaded as part of the project's default instructions is incorrect because default instructions do not create a reusable slash skill, do not enforce allowed-tools, and do not provide a forked execution context. This approach would also push analysis chatter into the main thread.

Create the skill under ~/.claude/skills/ with frontmatter that sets allowed-tools to Read, Grep, and Glob and uses context: fork is incorrect because a user home directory location is local to one machine and is not shared through the repository with other developers. It also does not follow the expected project layout with a SKILL.md inside .claude/skills/ that can be committed and reviewed.

Create a project command under .claude/commands/ and configure allowed-tools to Read, Grep, and Glob is incorrect because commands are not the same as skills and they do not run in a forked

context by default. The output would land in the main conversation and this does not meet the requirement to keep lengthy analysis out of the primary thread.

EXAM TIP

Map requirements directly to configuration. If it must be *shared* with all developers then look for repository paths like `.claude/skills/`. If it must stay *read only* then confirm `allowed-tools` excludes `write` or `exec`. If it must not clutter the main conversation then look for *context: fork*.

Question 15

In the team's Claude Code workflow, a custom architecture-review skill generates extensive discovery notes that overwhelm the main session before the developer can continue with the actual coding task. Which SKILL.md frontmatter setting should be used to keep that exploratory work isolated?

This question deals with the [Code Generation with Claude Code Exam Scenario](#).

(Choose 1)

argument-hint, to prompt the developer for the architecture area to inspect

context: fork, to run the skill in a separate sub-agent context ✓

allowed-tools, to limit the skill to read-only repository access

paths, to activate the skill only for architecture-related files

ANSWER

The correct choice is **context: fork, to run the skill in a separate sub-agent context** because it executes the exploratory work in an isolated branch of the conversation so the long discovery notes do not flood the main session and the developer can continue coding smoothly.

With **context: fork, to run the skill in a separate sub-agent context** the skill launches in its own sub-agent conversation that maintains a separate history. This keeps the primary chat clean and preserves the developer's focus. Only the intended outputs such as a concise summary or artifacts are returned to the caller while the bulk of the exploratory notes remain contained within the forked run. This is exactly what you need when discovery steps would otherwise consume the main context window and distract from the coding task.

argument-hint, to prompt the developer for the architecture area to inspect is incorrect because it only guides input collection and does not create an isolated execution context. It cannot prevent verbose notes from appearing in the main session.

allowed-tools, to limit the skill to read-only repository access is incorrect because tool permission scopes do not affect conversation routing or isolation. Limiting tools will not keep discovery notes out of the main thread.

paths, to activate the skill only for architecture-related files is incorrect because file path activation controls when the skill triggers and it does not determine where the conversation content is stored or displayed. It will not isolate the exploratory output.

EXAM TIP

When a scenario asks to keep exploratory output separate from the main session, look for keywords like *isolate* or *fork* and map them to the frontmatter that controls execution context.

Question 16

The customer support resolution agent has a system prompt that tells it to stay in a policy-grounded support coordinator role and use the MCP tools when needed. It follows this behavior in the first few exchanges, but by the eighth turn it starts giving broad generic advice, even though the conversation is only about 2,800 tokens. What is the most likely cause?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

System prompts define behavior only at the start of a session

The model's attention declines simply because more turns have occurred

Prior assistant messages have diluted the behavioral pattern from the system prompt ✓

The system prompt is not included after the first API request

ANSWER

The correct option is **Prior assistant messages have diluted the behavioral pattern from the system prompt**. The conversation is only about 2,800 tokens and the agent stayed on policy for several turns, which strongly indicates that the model began pattern matching against its own most recent assistant messages. That recent pattern can outweigh earlier steering and cause a gradual drift into generic advice even though the system prompt is still present.

This happens because the model conditions on the entire message sequence and strongly imitates the latest conversational patterns. If earlier assistant replies become slightly less policy grounded, the model can reinforce that trajectory on subsequent turns. In short conversations that do not approach the context limit, the most common cause of role drift is cumulative imitation of recent assistant behavior rather than loss or truncation of the system prompt.

System prompts define behavior only at the start of a session is incorrect. System messages apply whenever they are included in the request, and typical orchestrations resend the system prompt on every turn. Their influence does not end after the first message in a multi turn exchange.

The model's attention declines simply because more turns have occurred is incorrect. There is no inherent decay tied only to turn count. When the context is well within the window, changes in behavior are more plausibly due to recency effects and pattern imitation rather than a mechanical loss of attention over turns.

The system prompt is not included after the first API request is unlikely given the symptom pattern. If it were omitted after the first turn, you would typically see deviation much earlier. Many clients include the system prompt on every call, and with a conversation of only 2,800 tokens it is also very unlikely that the system prompt was truncated.

EXAM TIP

When diagnosing role drift in short chats, look for *recency and pattern imitation* before assuming truncation. Reinforce policy in assistant replies or periodically restate key constraints so the most recent examples keep the model anchored.

REFERENCES

Check out these links for more details:

- [Anthropic Messages API overview, including how to include system prompts and conversation history](#)
- [Anthropic guidance on system prompts and steering](#)
- [Anthropic prompt engineering best practices for conversation control and consistency](#)

Question 17

In the multi-agent research system, the web-search subagent retrieves competitor pages and industry reports but times out on news archives and social feeds, while the document analysis subagent completes its work. How should the synthesis subagent propagate the incomplete upstream results when drafting the research summary?

This question deals with the [Multi-Agent Research Exam Scenario](#).

(Choose 1)

Use the available search and document outputs to draft the summary without noting which source categories were missing

Return an error to the coordinator so the entire research task is retried or marked failed because some inputs are incomplete

Ask the coordinator to rerun the timed-out source categories with a longer timeout before any synthesis is attempted

Draft the synthesis with coverage notes that distinguish well-supported findings from gaps caused by unavailable sources ✓

ANSWER

The correct option is **Draft the synthesis with coverage notes that distinguish well-supported findings from gaps caused by unavailable sources.**

This approach is best because the synthesis subagent should preserve and leverage the high quality outputs it already has while making the state of evidence explicit. The web-search subagent returned competitor pages and industry reports, and the document analysis subagent completed, so there is enough signal to write a meaningful summary. By adding clear coverage notes that call out that news archives and social feeds were unavailable due to timeouts, the synthesis maintains traceability and communicates confidence levels. This lets stakeholders or the coordinator decide whether to accept the draft as is or to schedule targeted retries for the missing categories, and it avoids both premature failure and silent information gaps.

Use the available search and document outputs to draft the summary without noting which source categories were missing is incorrect because it hides important coverage gaps. This creates the risk of overstated conclusions and reduces auditability, since readers would not know that news archives and social feeds were not included.

Return an error to the coordinator so the entire research task is retried or marked failed because some inputs are incomplete is incorrect because it discards valuable completed work and hurts throughput. Errors should be reserved for cases where synthesis would be invalid without specific inputs, which is not true here since substantial evidence is available.

Ask the coordinator to rerun the timed-out source categories with a longer timeout before any synthesis is attempted is incorrect because it blocks progress and delays insights. A better pattern is to proceed with a draft that includes coverage notes, while the coordinator may schedule targeted retries in parallel if needed.

EXAM TIP

When inputs are partially complete, favor *coverage notes* and *traceability* over failure or silence. Preserve *partial results*, label gaps clearly, and let the workflow continue while missing pieces can be retried separately.

Question 18

While configuring subagents for the developer productivity agent, which statements describe how subagent definitions and context handling work?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 2)

A spawned subagent receives the coordinator agent's complete conversation history by default

A subagent's AgentDefinition specifies its role description, system prompt, and allowed tool set ✓

A coordinator can create subagents without enabling the Task tool when its prompt names the desired roles clearly

A subagent invocation has isolated context, so required background must be included explicitly in the subagent prompt ✓

Tool limits configured for a subagent serve only as guidance and are not applied during execution

ANSWER

The correct options are **A subagent's AgentDefinition specifies its role description, system prompt, and allowed tool set** and **A subagent invocation has isolated context, so required background must be included explicitly in the subagent prompt**.

A subagent's AgentDefinition specifies its role description, system prompt, and allowed tool set is correct because subagents are defined with a clear persona and scope. The definition captures what the subagent is supposed to do, how it should behave through its system prompt, and exactly which tools it is permitted to use. This enforces least privilege and ensures predictable behavior during coordination.

A subagent invocation has isolated context, so required background must be included explicitly in the subagent prompt is correct because subagents do not automatically inherit the coordinator's running history. Each invocation runs with only the context you supply at call time, so you must pass any prior decisions, constraints, artifacts, or user requirements in the subagent's instructions or inputs to ensure it can perform the task correctly.

A spawned subagent receives the coordinator agent's complete conversation history by default is incorrect because subagent calls are isolated from the coordinator's conversation by design. Only the content that the coordinator includes in the invocation is visible to the subagent.

A coordinator can create subagents without enabling the Task tool when its prompt names the desired roles clearly is incorrect because you need an explicit orchestration mechanism to create and run subagents. Simply naming roles in a prompt does not instantiate separate agents with their own tool permissions or execution boundaries.

Tool limits configured for a subagent serve only as guidance and are not applied during execution is incorrect because the allowed tool set and any related limits are enforced at runtime. The subagent cannot invoke tools it has not been granted, which protects against unintended actions and keeps each subagent constrained to its role.

EXAM TIP

When evaluating multi-agent questions, look for phrases like *isolated context* and *allowed tools*. If context isolation is mentioned, expect that you must *explicitly* pass background, and if tools are listed in a definition, assume those permissions are *enforced* not advisory.

REFERENCES

Check out these links for more details:

- [Anthropic documentation: Tool use](#)
- [Anthropic API: Messages](#)

Question 19

The customer support resolution agent calls process_refund, but the request is outside the allowed return window and the backend denies it. Which MCP tool error response best lets the agent continue the conversation appropriately with the customer?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Return errorCategory: "business", isRetryable: false, and a customer-ready explanation of the return-window policy ✓

Return errorCategory: "transient" and mark the failure as retryable in case the policy result changes later

Return a successful empty result so the agent can avoid surfacing a negative outcome to the customer

Return the backend stack trace so the agent has the most detailed diagnostic information available

ANSWER

The correct response is **Return errorCategory: "business", isRetryable: false, and a customer-ready explanation of the return-window policy**. This best supports the agent because it clearly identifies a policy-driven denial that will not succeed if attempted again and it provides a concise message the agent can relay to the customer.

A business rule failure is not a system fault and it is not temporary. Marking it as not retryable prevents wasted tool calls and guides the agent to pivot to the next helpful action such as offering store credit or explaining exceptions if any exist. Providing a short customer ready explanation equips the agent with precise wording that reduces hesitation and keeps the conversation empathetic and clear. This structure also helps orchestration layers decide not to auto retry and to surface the explanation directly in the agent workflow.

Return errorCategory: "transient" and mark the failure as retryable in case the policy result changes later is incorrect because a return window denial is not a temporary issue. Transient errors indicate conditions like network instability, timeouts, or rate limits that could succeed on another attempt. Marking a firm policy as retryable misleads the agent and may trigger pointless retries.

Return a successful empty result so the agent can avoid surfacing a negative outcome to the customer is incorrect because it hides the true outcome and forces the agent to guess or fabricate next steps. Tools must return accurate states so the agent can communicate clearly and maintain trust.

Return the backend stack trace so the agent has the most detailed diagnostic information available is incorrect because this is not a system error and a stack trace is not customer friendly. Exposing

internals risks leaking sensitive details and does not help the agent explain a policy decision to the customer.

EXAM TIP

First decide whether the error is a *business* rule outcome or a *transient* system issue. Set *isRetryable* to false for policy denials and include a short *customer ready* explanation the agent can use verbatim. Reserve transient with retries for temporary faults like timeouts or rate limits.

REFERENCES

Check out these links for more details:

- [Anthropic docs: Tool use with Claude](#)
- [Model Context Protocol specification](#)
- [Model Context Protocol specification repository](#)

Question 20

During a long customer support interaction, the resolution agent's context has grown to 76,000 tokens with customer identifiers, order details, refund amounts, policy clarifications, and general back-and-forth. The team needs to reduce context size without losing information needed to resolve or escalate the case. What approach best balances token reduction with preserving important support details?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Condense the entire transcript into a single conversation summary

Retain only the most recent 20,000 tokens from the conversation

Extract critical case facts into a structured block, summarize general discussion, and keep recent exchanges verbatim ✓

Save the full transcript outside the prompt and retrieve relevant sections with semantic search

ANSWER

The correct choice is **Extract critical case facts into a structured block, summarize general discussion, and keep recent exchanges verbatim**. This balances token reduction with preservation of essential details by compressing what can be safely summarized while keeping the most action-relevant and time-sensitive parts intact.

This approach works because it separates information by utility. A compact structured facts block preserves identifiers, order numbers, dollar amounts, policy citations, decisions, and deadlines in a machine friendly format that is easy for the model to reference with minimal tokens. Summarizing the general back and forth removes filler and repetition while retaining the gist of what happened. Keeping the most recent turns verbatim preserves current intent, the latest instructions, and any unresolved questions, which are often needed to formulate the next response or decide on escalation. Together these layers lower token usage while maintaining fidelity for both resolution and auditability.

Condense the entire transcript into a single conversation summary is not ideal because a single summary tends to blur precise facts such as customer identifiers, exact refund amounts, and verbatim commitments. It also removes quotations and nuanced wording that can matter in support decisions and escalations.

Retain only the most recent 20,000 tokens from the conversation risks discarding earlier critical context like account verification steps, policy clarifications, prior approvals, or case numbers that are often introduced early. Recency alone does not ensure completeness for a support resolution.

Save the full transcript outside the prompt and retrieve relevant sections with semantic search can be useful in some architectures, yet it adds retrieval complexity and the chance of missing key context

during a live case handoff. For a single ongoing interaction it is typically more reliable to keep a structured facts block and a concise summary with recent turns verbatim inside the prompt so the model has guaranteed access to what it needs at response time.

EXAM TIP

When trimming long conversations, first pull out *facts* into a compact structured block, then write a short *summary* for general chatter, and keep the most *recent* exchanges verbatim. This pattern reduces tokens while preserving what drives the next action.

REFERENCES

Check out these links for more details:

- [Amazon Bedrock prompt engineering best practices](#)
- [AWS What is Retrieval Augmented Generation](#)